

Back to Basics: Let Denoising Generative Models Denoise

Tianhong Li Kaiming He
MIT

Abstract

Today’s denoising diffusion models do not “denoise” in the classical sense, i.e., they do not directly predict clean images. Rather, the neural networks predict noise or a noised quantity. In this paper, we suggest that predicting clean data and predicting noised quantities are fundamentally different. According to the manifold assumption, natural data should lie on a low-dimensional manifold, whereas noised quantities do not. With this assumption, we advocate for models that directly predict clean data, which allows apparently under-capacity networks to operate effectively in very high-dimensional spaces. We show that simple, large-patch Transformers on pixels can be strong generative models: using no tokenizer, no pre-training, and no extra loss. Our approach is conceptually nothing more than “**Just image Transformers**”, or **JiT**, as we call it. We report competitive results using JiT with large patch sizes of 16 and 32 on ImageNet at resolutions of 256 and 512, where predicting high-dimensional noised quantities can fail catastrophically. With our networks mapping back to the basics of the manifold, our research goes back to basics and pursues a self-contained paradigm for Transformer-based diffusion on raw natural data.

1. Introduction

When diffusion generative models were first developed [57, 59, 23], the core idea was supposed to be *denoising*, i.e., predicting a clean image from its corrupted version. However, two significant milestones in the evolution of diffusion models turned out to deviate from the goal of directly predicting clean images. First, predicting the noise itself (known as “ ϵ -prediction”) [23] made a pivotal difference in generation quality and largely popularized these models. Later, diffusion models were connected to flow-based methods [37, 38, 1] through predicting the flow velocity (“ v -prediction” [52]), a quantity that combines clean data and noise. Today, diffusion models in practice commonly predict noise or a noised quantity (e.g., velocity).

Extensive studies [52, 29, 25, 15] have shown that pre-

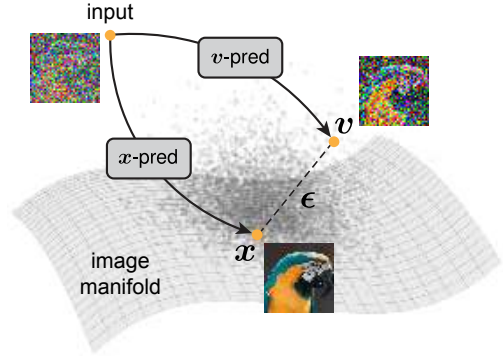


Figure 1. **The Manifold Assumption** [4] hypothesizes that natural images lie on a low-dimensional manifold within the high-dimensional pixel space. While a clean image x can be modeled as on-manifold, the noise ϵ or flow velocity v (e.g., $v = x - \epsilon$) is inherently off-manifold. Training a neural network to predict a clean image (i.e., x -prediction) is fundamentally different from training it to predict noise or a noised quantity (i.e., ϵ/v -prediction).

dicting the clean image (“ x -prediction” [52]) is closely related to ϵ - and v -prediction, provided that the *weighting* of the prediction loss is reformulated accordingly (detailed in Sec. 3). Owing to this relation, less attention has been paid to what the network should directly predict, implicitly assuming that the network is capable of performing the assigned task.

However, the roles of clean images and a noised quantity (including the noise itself) are *not* equal. In machine learning, it has long been hypothesized [4, 3] that “(high-dimensional) data lie (roughly) on a low-dimensional manifold” ([4, p.6]). Under this manifold assumption, while clean data can be modeled as lying on a low-dimensional manifold, a noised quantity is inherently distributed across the full high-dimensional space [69] (see Fig. 1). *Predicting clean data is fundamentally different from predicting noise or a noised quantity.*

Consider a scenario where a low-dimensional manifold is embedded in a high-dimensional observation space. Predicting noise in this high-dimensional space requires *high capacity*: the network needs to preserve all information about the noise. In contrast, a *limited-capacity* network can

still predict the clean data, as it only needs to retain the low-dimensional information while filtering out the noise.

When a low-dimensional space (e.g., image latent [49]) is used, the difficulty of predicting noise is alleviated, yet at the same time is *hidden* rather than solved. When it comes to pixel or other high-dimensional spaces, existing diffusion models can still struggle to address the *curse of dimensionality* [4]. The heavy reliance on a pre-trained latent space prevents diffusion models from being self-contained.

In pursuit of a self-contained principle, there has been strong focus on advancing diffusion modeling in the pixel space [7, 25, 26, 6, 70]. In general, these methods explicitly or implicitly avoid the information bottleneck in the networks, e.g., by using dense convolutions or smaller patches, increasing channels, or adding long skip connections. We suggest that these designs may stem from the demand to predict high-dimensional noised quantities.

In this paper, we return to first principles and let the neural network *directly* predict the clean image. By doing so, we show that a *plain* Vision Transformer (ViT) [13], operating on large image patches consisting of raw pixels, can be effective for diffusion modeling. Our approach is *self-contained* and does not rely on any pre-training or auxiliary loss — *no latent tokenizer* [49], *no adversarial loss* [16, 49], *no perceptual loss* [77, 49] (thus *no pre-trained classifier* [56]), and *no representation alignment* [74] (thus *no self-supervised pre-training* [45]). Conceptually, our model is nothing more than “Just image Transformers”, or **JiT**, as we call it, applied to diffusion.

We conduct experiments on the ImageNet dataset [11] at resolutions of 256 and 512, using JiT models with patch sizes 16 and 32 respectively. Even though the patches are very high-dimensional (hundreds or thousands), our models using x -prediction can easily produce strong results, where ϵ - and v -prediction fail catastrophically. Further analysis shows that it is unnecessary for the network width to match or exceed the patch dimension; in fact, and surprisingly, a *bottleneck* design can even be beneficial, echoing observations from classical manifold learning.

Our effort marks a step toward a self-contained “Diffusion + Transformer” philosophy [46] on native data. Beyond computer vision, this philosophy is highly desirable in other domains involving natural data (e.g., proteins, molecules, or weather), where a tokenizer can be difficult to design. By minimizing domain-specific designs, we hope that the general “Diffusion + Transformer” paradigm originated from computer vision will find broader applicability.

2. Related Work

Diffusion Models and Their Predictions. The pioneering work on diffusion models [57] proposed to learn a reversed stochastic process, in which the network predicts the pa-

rameters of a normal distribution (e.g., mean and standard deviation). Five years after its introduction, this method was revitalized and popularized by Denoising Diffusion Probabilistic Models (DDPM) [23]: a *pivotal* discovery was to make noise the prediction target (i.e., ϵ -prediction).

The relationships among different prediction targets were then investigated in [52] (originally in the context of model distillation), where the notion of v -prediction was also introduced. Their work focused on the weighting effects introduced by reparameterization.

Meanwhile, EDM [29] reformulated the diffusion problem around a *denoiser function*, which constitutes a major milestone in the evolution of diffusion models. However, EDM adopted a *pre-conditioned* formulation, where the direct output of the network is *not* the denoised image. While this formulation is preferable in low-dimensional scenarios, it still inherently requires the network to output a quantity that mixes data and noise (more comparisons in appendix).

Flow Matching models [37, 38, 1] can be interpreted as a form of v -prediction [52] within the diffusion modeling framework. Unlike pure noise, v is a combination of data and noise. The connections between flow-based models and previous diffusion models have been established [15]. Today, diffusion models and their flow-based counterparts are often studied under a unified framework.

Denoising Models. Over the past decades, the concept of denoising has been closely related to representation learning. Classical methods, exemplified by BM3D [9] and others [47, 14, 79], leveraged the assumptions of sparsity and low dimensionality to perform image denoising.

Denoising Autoencoders (DAEs) [68, 69] were developed as an unsupervised representation learning method, using denoising as their training objective. They leveraged the manifold assumption [4] (Fig. 1) to learn meaningful representations that approximate the low-dimensional data manifold. DAEs can be viewed as a form of Denoising Score Matching [67], which in turn is closely related to modern score-based diffusion models [59, 60]. Nevertheless, while it is natural for DAEs to predict clean data for manifold learning, in score matching, predicting the score function effectively amounts to predicting the noise (up to a scaling factor), i.e., ϵ -prediction.

Manifold Learning. Manifold learning has been a classical field [51, 63] focused on learning low-dimensional, nonlinear representations from observed data. In general, manifold learning methods leverage *bottleneck* structures [64, 48, 41, 2] that encourage only useful information to pass through. Several studies have explored the connections between manifold learning and generative models [39, 27, 8]. Latent Diffusion Models (LDMs) [49] can be viewed as manifold learning in the first stage via an autoencoder, followed by diffusion in the second stage.

Pixel-space Diffusion. While latent diffusion [49] has become the default choice in the field today, the development of diffusion models originally began with pixel-space formulations [59, 23, 44, 12]. Early pixel-space diffusion models were typically based on dense convolutional networks, most often a U-Net [50]. These models often use over-complete channel representations (e.g., transforming $H \times W \times 3$ into $H \times W \times 128$ in the first layer), accompanied by long-range skip connections. While these models work well for ϵ - and v -prediction, their dense convolutional structures are typically computationally expensive. Applying these convolutional models to high-resolution images does not lead to catastrophic degradation, and as such, research in this direction has commonly focused on noise schedules and/or weighting schemes [7, 25, 26, 30] to further improve generation quality.

In contrast, applying a Vision Transformer (ViT) [13] directly to pixels presents a more challenging task. *Standard* ViT architectures adopt an aggressive patch size (e.g., 16×16 pixels), resulting in a high-dimensional token space that can be comparable to, or larger than, the Transformer’s hidden dimension. SiD2 [26] and PixelFlow [6] adopt hierarchical designs that start from smaller patches; however, these models are “FLOP-heavy” [26] and lose the inherent generality and simplicity of standard Transformers. PixNerd [70] adopts a NeRF head [43] that integrates information from the Transformer output, noisy input, and spatial coordinates, with training further assisted by representation alignment [74].

Even with these special-purpose designs, the architectures in these works typically start from the “L” (Large) or “XL” size. In fact, a latest work [73] suggests that a large hidden size appears necessary for high dimensionality.

High-dimensional Diffusion. When using ViT-style architectures, modern diffusion models are still challenged by high-dimensional input spaces, whether in pixels or latents. In the literature [8, 73, 55], it has been *repeatedly* reported that ViT-style diffusion models degrade rapidly and *catastrophically* when the per-token dimensionality increases, regardless of the use of pixels or latents.

Concurrently with our work, a line of research [78, 34, 55] resorts to *self-supervised pre-training* to address high-dimensional diffusion. In contrast to these efforts, we show that high-dimensional diffusion is achievable *without* any pre-training, and using *just* Transformers.

x -prediction. The formulation of x -prediction is natural and not new; it can be traced back at least to the original DDPM [23] (see their code [24]). However, DDPM observed that ϵ -prediction was substantially better, which later became the go-to solution. In later works (e.g., [26]), although the analysis was sometimes preferably conducted in the x -space, the actual prediction was often made in other spaces, likely for legacy reasons.

When it comes to the image restoration application addressed by diffusion [10, 72, 42], it is natural for the network to predict the clean data, as this is the ultimate goal of image restoration. Concurrent with our work, [18] also advocates the use of x -prediction, for generative world models that are conditional on previous frames.

Our work does not aim to reinvent this fundamental concept; rather, we aim to draw attention to a largely overlooked yet critical issue in the context of high-dimensional data with underlying low-dimensional manifolds.

3. On Prediction Outputs of Diffusion Models

Diffusion models can be formulated in the space of x , ϵ , or v . The choice of the space determines not only where the loss is defined, but also what the network predicts. Importantly, *the loss space and the network output space need not be the same*. This choice can make critical differences.

3.1. Background: Diffusion and Flows

Diffusion models can be formulated from the perspective of ODEs [5, 60, 37, 58]. We begin our formulation with the flow-based paradigm [37, 38, 1], i.e., in the v -space, as a simpler starting point, and then discuss other spaces.

Consider a data distribution $x \sim p_{\text{data}}(x)$ and a noise distribution $\epsilon \sim p_{\text{noise}}(\epsilon)$ (e.g., $\epsilon \sim \mathcal{N}(0, \mathbf{I})$). During training, a noisy sample z_t is an interpolation: $z_t = a_t x + b_t \epsilon$, where a_t and b_t are pre-defined noise schedules at time $t \in [0, 1]$. In this paper, we use a linear schedule [37, 38, 1]¹: $a_t = t$ and $b_t = 1 - t$. This gives:

$$z_t = t x + (1 - t) \epsilon, \quad (1)$$

which leads to $z_t \sim p_{\text{data}}$ when $t=1$. We use the logit-normal distribution over t [15], i.e., $\text{logit}(t) \sim \mathcal{N}(\mu, \sigma^2)$.

A flow velocity v is defined as the time-derivative of z , that is, $v_t = z'_t = a'_t x + b'_t \epsilon$. Given Eq. (1), we have:

$$v = x - \epsilon. \quad (2)$$

The flow-based methods [37, 38, 1] minimize a loss function defined as:

$$\mathcal{L} = \mathbb{E}_{t,x,\epsilon} \|v_\theta(z_t, t) - v\|^2, \quad (3)$$

where v_θ is a function parameterized by θ . While v_θ is often the *direct* output of a network $v_\theta = \text{net}_\theta(z_t, t)$ [37, 38, 1], it can also be a transform of it, as we will elaborate.

Given the function v_θ , sampling is done by solving an ordinary differential equation (ODE) for z [37, 38, 1]:

$$dz_t/dt = v_\theta(z_t, t), \quad (4)$$

starting from $z_0 \sim p_{\text{noise}}$ and ending at $t = 1$. In practice, this ODE can be approximately solved using numerical solvers. By default, we use a 50-step Heun.

¹Our analysis in this paper is applicable to other schedules.

	(a) x -pred $\mathbf{x}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$	(b) ϵ -pred $\mathbf{\epsilon}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$	(c) v -pred $\mathbf{v}_\theta := \text{net}_\theta(\mathbf{z}_t, t)$
(1) x -loss: $\mathbb{E}\ \mathbf{x}_\theta - \mathbf{x}\ ^2$	$\mathbf{x}_\theta$	$\mathbf{x}_\theta = (\mathbf{z}_t - (1-t)\mathbf{\epsilon}_\theta)/t$	$\mathbf{x}_\theta = (1-t)\mathbf{v}_\theta + \mathbf{z}_t$
(2) ϵ -loss: $\mathbb{E}\ \mathbf{\epsilon}_\theta - \mathbf{\epsilon}\ ^2$	$\mathbf{\epsilon}_\theta = (\mathbf{z}_t - t\mathbf{x}_\theta)/(1-t)$	$\mathbf{\epsilon}_\theta$	$\mathbf{\epsilon}_\theta = \mathbf{z}_t - t\mathbf{v}_\theta$
(3) v -loss: $\mathbb{E}\ \mathbf{v}_\theta - \mathbf{v}\ ^2$	$\mathbf{v}_\theta = (\mathbf{x}_\theta - \mathbf{z}_t)/(1-t)$	$\mathbf{v}_\theta = (\mathbf{z}_t - \mathbf{\epsilon}_\theta)/t$	$\mathbf{v}_\theta$

Table 1. **All possible combinations** of defining the loss and network prediction in x , v , or ϵ spaces. The direct network outputs are highlighted in colors. For any off-diagonal entry where the network output space differs from the loss space, a transformation on the network output is applied.

3.2. Prediction Space and Loss Space

Prediction Space. The network’s direct output can be defined in any space: v , x , or ϵ . Next, we discuss the resulting transformation. Note that in the context of this paper, we refer to it as “ x , ϵ , v -prediction”, *only* when the network net_θ ’s *direct* output is strictly x , ϵ , v , respectively.

Given three unknowns (x , ϵ , v) and one network output, we require two additional constraints to determine all three unknowns. The two constraints are given by Eq. (1) and (2). For example, when we let the direct network output net_θ be x , we solve the following set of equations:

$$\begin{cases} \mathbf{x}_\theta = \text{net}_\theta \\ \mathbf{z}_t = t\mathbf{x}_\theta + (1-t)\mathbf{\epsilon}_\theta \\ \mathbf{v}_\theta = \mathbf{x}_\theta - \mathbf{\epsilon}_\theta \end{cases} \quad (5)$$

Here, the notations $\mathbf{x}_\theta$, $\mathbf{\epsilon}_\theta$, and $\mathbf{v}_\theta$ suggest that they are all predictions dependent on θ . Solving this equation set gives: $\mathbf{\epsilon}_\theta = (\mathbf{z}_t - t\mathbf{x}_\theta)/(1-t)$ and $\mathbf{v}_\theta = (\mathbf{x}_\theta - \mathbf{z}_t)/(1-t)$, that is, both $\mathbf{\epsilon}_\theta$ and $\mathbf{v}_\theta$ can be computed from $\mathbf{z}_t$ and the network $\mathbf{x}_\theta$. These are summarized in Tab. 1 in column (a).

Similarly, when we let the direct network output net_θ be ϵ or v , we obtain the other sets of equations (by replacing the first one in Eq. (5)). The transformations are summarized in Tab. 1 in the columns of (b), (c) for ϵ -, v -prediction. This shows that when one quantity in $\{x, \epsilon, v\}$ is predicted, the other two can be inferred. The derivations in many prior works (e.g., [52, 15]) are special cases covered in Tab. 1.

Loss Space. While the loss is often defined in one reference space (e.g., v -loss in Eq. (3)), conceptually, one can define it in any space. It has been shown [52, 15] that with a given reparameterization from one prediction space to another, the loss is effectively reweighted.

For example, consider the combination of x -prediction and v -loss in Tab. 1(3)(a). We have $\mathbf{v}_\theta = (\mathbf{x}_\theta - \mathbf{z}_t)/(1-t)$ as the prediction and $\mathbf{v} = (\mathbf{x} - \mathbf{z}_t)/(1-t)$ as the target. The v -loss in Eq. (3) becomes: $\mathcal{L} = \mathbb{E}\|\mathbf{v}_\theta(\mathbf{z}_t, t) - \mathbf{v}\|^2 = \mathbb{E}\frac{1}{(1-t)^2}\|\mathbf{x}_\theta(\mathbf{z}_t, t) - \mathbf{x}\|^2$, which is a *reweighted* form of the x -loss. A transformation like this one can be done for any prediction space and any loss space listed in Tab. 1.

Put together, consider the three *unweighted* losses defined in $\{x, \epsilon, v\}$, and the three forms of the network direct output, there are *nine possible combinations* (Tab. 1). Each combination constitutes a valid formulation, and *no*

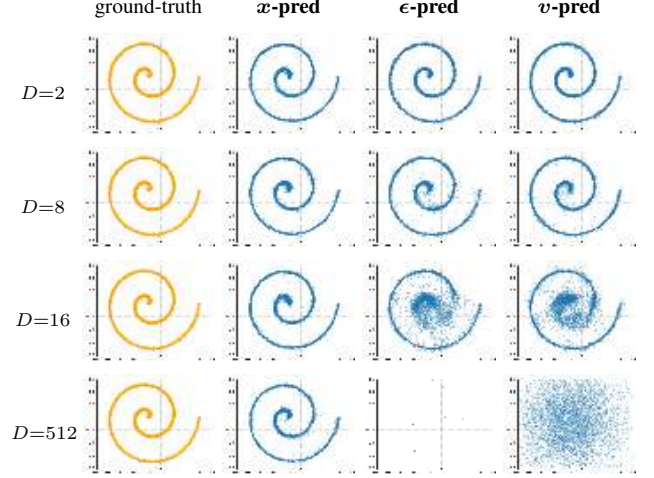


Figure 2. **Toy Experiment:** d -dimensional ($d = 2$) underlying data is “buried” in a D -dimensional space, by a fixed, random, column-orthogonal projection matrix. In the D -dim space, we train a simple generative model (5-layer ReLU MLP with 256-dim hidden units). The projection matrix is unknown to the model, and we only use it for visualizing the output. In this toy experiment, with the observed dimension D increasing, only x -prediction can produce reasonable results.

two among the nine cases are mathematically equivalent.

Generator Space. Regardless of the combination used, to perform generation at inference-time, we can always transform the network output to the v -space (Tab. 1, row (3)) and solve the ODE in Eq. (4) for sampling. As such, all nine combinations are legitimate generators.

3.3. Toy Experiment

According to the manifold assumption [4], the data x tends to lie in a low-dimensional manifold (Fig. 1), while noise ϵ and velocity v are off-manifold. Letting a network directly predict the clean data x should be more tractable. We verify this assumption in a toy experiment in this section.

We consider the toy case of d -dimensional underlying data “buried” in an observed D -dimensional space ($d < D$). We synthesize this scenario using a projection matrix $P \in \mathbb{R}^{D \times d}$ that is column-orthogonal, i.e., $P^\top P = I_{d \times d}$. This matrix P is randomly created and fixed. The observed data is $\mathbf{x} = P\hat{\mathbf{x}} \in \mathbb{R}^D$, where the underlying data is $\hat{\mathbf{x}} \in \mathbb{R}^d$. The matrix P is unknown to the model, and as such, it is a D -dimensional generation problem for the model.

We train a 5-layer ReLU MLP with 256-dim hidden units as the generator and visualize the results in Fig. 2. We obtain these visualizations by projecting the D -dim generated samples back to d -dim using P . We investigate the cases of $D \in \{2, 8, 16, 512\}$ for $d = 2$. We study $\mathbf{x}$, ϵ , or $\mathbf{v}$ -prediction, all using the $\mathbf{v}$ -loss, *i.e.*, Tab. 1(3)(a-c).

Fig. 2 shows that *only $\mathbf{x}$ -prediction can produce reasonable results when D increases*. For ϵ -/ $\mathbf{v}$ -prediction, the models struggle at $D=16$, and fail catastrophically when $D=512$, where the 256-dim MLP is under-complete.

Notably, *$\mathbf{x}$ -prediction can work well even when the model is under-complete*. Here, the 256-dim MLP inevitably discards information in the $D=512$ -dim space. However, since the true data is in a low-dimensional d -dim space, $\mathbf{x}$ -prediction can still perform well, as the ideal output is implicitly d -dim. We draw similar observations in the case of real data on ImageNet, as we show next.

4. “Just Image Transformers” for Diffusion

Driven by the above analysis, we show that *plain* Vision Transformers (ViT) [13] operating on pixels can work surprisingly well, simply using $\mathbf{x}$ -prediction.

4.1. Just Image Transformers

The core idea of ViT [13] is “*Transformer on Patches (ToP)*”². Our architecture design follows this philosophy.

Formally, consider $H \times W \times C$ -dim image data ($C=3$). All $\mathbf{x}$, ϵ , $\mathbf{v}$ and $\mathbf{z}_t$ share this same dimensionality. Given an image, we divide it into non-overlapping patches of size $p \times p$, resulting in a sequence of a length $\frac{H}{p} \times \frac{W}{p}$. Each patch is a $p \times p \times 3$ -dim vector. This sequence is processed by a linear embedding projection, added with positional embedding [66], and mapped by a stack of Transformer blocks [66]. The output layer is a linear predictor that projects each token back to a $p \times p \times 3$ -dim patch. See Fig. 3.

As standard practice, the architecture is conditioned on time t and a given class label. We use adaLN-Zero [46] for conditioning and will discuss other options later. Conceptually, this architecture amounts to the Diffusion Transformer (DiT) [46] directly applied to patches of pixels.

The overall architecture is nothing more than “Just image Transformers”, which we refer to as **JiT**. For example, we investigate **JiT/16** (*i.e.*, patch size $p=16$, [13]) on 256×256 images, and **JiT/32** ($p=32$) on 512×512 images. These settings respectively result in a dimensionality of 768 ($16 \times 16 \times 3$) and 3072 ($32 \times 32 \times 3$) per patch. Such high-dimensional patches can be handled by $\mathbf{x}$ -prediction.

4.2. What to Predict by the Network?

We have summarized the nine possible combinations of loss space and prediction space in Tab. 1. For each of these com-

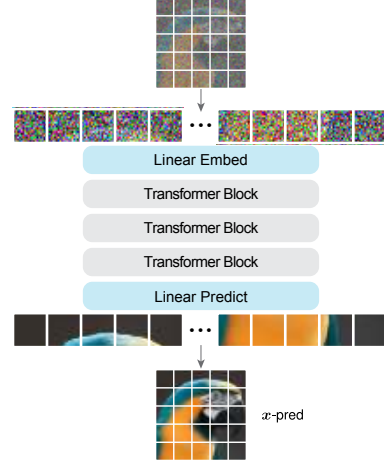


Figure 3. The “Just image Transformer” (JiT) architecture: simply a plain ViT [13] on patches of pixels for $\mathbf{x}$ -prediction.

binations, we train a “Base” [13] model (JiT-B), which has a hidden size of 768-dim per token. We study JiT-B/16 at 256×256 resolution in Tab. 2(a). As a reference, we examine JiT-B/4 (*i.e.*, $p=4$) at 64×64 in Tab. 2(b). In both settings, the sequence length is the same (16×16).

We draw the following observations:

$\mathbf{x}$ -prediction is critical. In Tab. 2(a) with JiT-B/16, *only $\mathbf{x}$ -prediction performs well*, and it works across all three losses. Here, a patch is 768-d ($16 \times 16 \times 3$), which coincides with the hidden size of 768 in JiT-B. While this may seem “about enough”, in practice the models may require additional capacity, *e.g.*, to handle positional embeddings. For ϵ -/ $\mathbf{v}$ -prediction, the model does not have enough capacity to separate and retain the noised quantities. These observations are similar to those in the toy case (Fig. 2).

As a comparison, we examine JiT-B/4 at 64×64 resolution (Tab. 2(b)). Here, all cases perform reasonably well: the accuracy gaps among the nine combinations are marginal, not decisive. The dimensionality is 48 ($4 \times 4 \times 3$) per patch, well below the hidden size of 768 in JiT-B, which explains why all combinations work reasonably well. We note that many previous *latent* diffusion models have a similarly small input dimensionality and therefore were not exposed to the issue we discuss here.

Loss weighting is not sufficient. Our work is not the first to enumerate the combinations of relevant factors. In [52], it has explored the combinations of loss *weighting* and network predictions. Their experiments were done on the low-dimensional CIFAR-10 dataset, using a U-net. Their observations were closer to ours on ImageNet 64×64 .³

However, Tab. 2(a) on ImageNet 256×256 suggests that *loss weighting is not the whole story*. On one hand, both

²Quoting Lucas Bayer.

³In the CIFAR-10 experiments in [52], they enumerated three types of loss weighting and three types of network outputs. In 8 out of these 9 combinations, their models work reasonably well (see their Tab. 1).

	x -pred	ϵ -pred	v -pred
x -loss	10.14	379.21	107.55
ϵ -loss	10.45	394.58	126.88
v -loss	8.62	372.38	96.53

(a) ImageNet 256×256, JiT-B/16

	x -pred	ϵ -pred	v -pred
x -loss	5.76	6.20	6.12
ϵ -loss	3.56	4.02	3.76
v -loss	3.55	3.63	3.46

(b) ImageNet 64×64, JiT-B/4

Table 2. **Results of all combinations** of loss space and network space (see Tab. 1), evaluated by FID-50K on ImageNet: (a) JiT-B/16 at 256 resolution, 768-d per patch; (b) JiT-B/4 at 64 resolution, 48-d per patch. We annotate **catastrophic failures** in red and **reasonable results** by green. Settings: 200 epochs, with CFG [22].

ϵ - and v -prediction fail catastrophically in Tab. 2(a), regardless of the loss space, which corresponds to different effective weightings in different loss spaces (as discussed). On the other hand, x -prediction works across *all* three loss spaces: the loss weighting induced by the v -loss is preferable, but not critical.⁴

Noise-level shift is not sufficient. Prior works [7, 25, 26] have suggested that increasing the noise level is useful for high-resolution pixel-based diffusion. We examine this in Tab. 3 with JiT-B/16. As we use the logit-normal distribution [15] for sampling t (see appendix), the noise level can be shifted by changing the parameter μ of this distribution: intuitively, shifting μ towards the negative side results in smaller t and thus increases the noise level (Eq. (1)).

Tab. 3 shows that when the model already performs decently (here, x -pred), appropriately high noise is beneficial, which is consistent with prior observations [7, 25, 26]. However, adjusting the noise level *alone* cannot remedy ϵ - or v -prediction: their failure stems inherently from the inability to propagate high-dimensional information.

As a side note, according to Tab. 3, we set $\mu = -0.8$ in other experiments on ImageNet 256×256.

Increasing hidden units is not necessary. Since the capacity can be limited by the network width (*i.e.*, numbers of hidden units), a natural idea is to increase it. However, this remedy is neither principled nor feasible when the observed dimension is very high. We show that this is not necessary in the case of x -prediction.

In Tab. 5 and Tab. 6 in the next section, we show results of **JiT/32** at resolution 512 and **JiT/64** at resolution 1024, using a *proportionally large* patch size of $p=32$ or $p=64$. This amounts to 3072-dim (*i.e.*, $32 \times 32 \times 3$) or 12288-dim per patch, substantially larger than the hidden size of B, L, and H models (defined in [13]). Nevertheless, x -prediction

⁴From Tab. 1(a), we see that with x -prediction, the coefficients of x_θ are 1, $t/(1-t)$, and $-1/(1-t)$ in the three rows. When converting to x -loss, the weights of x -loss are 1, $t^2/(1-t)^2$, and $1/(1-t)^2$, respectively.

t -shift (μ)	x -pred	ϵ -pred	v -pred
(lower noise) 0.0	14.44	464.25	120.03
-0.4	9.79	372.91	109.93
-0.8	8.62	372.36	96.53
(higher noise) -1.2	8.99	355.25	106.85

Table 3. **Noise-level shift** (JiT-B/16, ImageNet 256×256, FID-50K). We shift the noise level by adjusting μ in the logit-normal t -sampler [15]. An appropriate noise level is useful, but is not sufficient for addressing the catastrophic failure in ϵ -/ v -prediction. Settings (the same as Tab. 2): 200 epochs, with CFG.

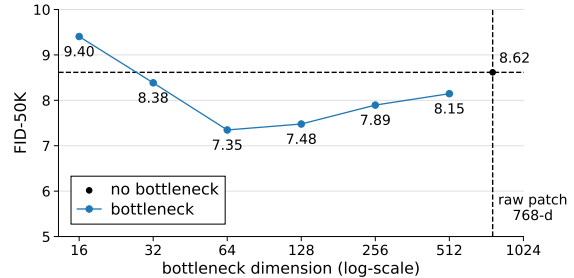


Figure 4. **Bottleneck linear embedding.** Results are for JiT-B/16 on ImageNet 256×256. A raw patch is 768-dim ($16 \times 16 \times 3$) and is embedded by two sequential linear layers with an intermediate bottleneck dimension d' ($d' < 768$). Here, **bottleneck embedding is generally beneficial**, and our x -prediction model can work decently even with aggressive bottlenecks as small as 32 or 16. Settings (the same as Tab. 3): 200 epochs, with CFG.

works well; in fact, it works *without* any modification other than scaling the noise proportionally (*e.g.*, by $2\times$ and $4\times$ at resolution 512 and 1024; see appendix).

This evidence suggests that the network design can be largely *decoupled* from the observed dimensionality, as is the case in many other neural network applications. Increasing the number of hidden units can be beneficial (as widely observed in deep learning), but it is not decisive.

Bottleneck can be beneficial. Even more surprisingly, we find that, conversely, introducing a bottleneck that *reduces* dimensionality in the network can be beneficial.

Specifically, we turn the linear patch embedding layer into a *low-rank* linear layer, by replacing it with a pair of bottleneck (yet still linear) layers. The first layer reduces the dimension to d' , and the second layer expands it to the hidden size of the Transformer. Both layers are linear and serve as a low-rank reparameterization.

Fig. 4 plots the FID *vs.* the bottleneck dimension d' , using JiT-B/16 (768-d per raw patch). Reducing the bottleneck dimension, even to as small as 16-d, does not cause catastrophic failure. In fact, a bottleneck dimension across a wide range (32 to 512) can improve the quality, by a decent margin of up to ~ 1.3 FID.

From a broader perspective of representation learning, this observation is not entirely unexpected. Bottleneck designs are often introduced to encourage the learning of inherently low-dimensional representations [64, 48, 41, 2].

Algorithm 1 Training step

```

# net(z, t): JiT network
# x: training batch

t = sample_t()
e = randn_like(x)

z = t * x + (1 - t) * e
v = (x - z) / (1 - t)

x_pred = net(z, t)
v_pred = (x_pred - z) / (1 - t)

loss = l2_loss(v - v_pred)

```

Algorithm 2 Sampling step (Euler)

```

# z: current samples at t

x_pred = net(z, t)
v_pred = (x_pred - z) / (1 - t)

z_next = z + (t_next - t) * v_pred

```

4.3. Our Algorithm

Our final algorithm adopts x -prediction and v -loss, which corresponds to Tab. 1(3)(a). Formally, we optimize:

$$\mathcal{L} = \mathbb{E}_{t,x,\epsilon} \left\| v_{\theta}(z_t, t) - v \right\|^2, \quad (6)$$

where: $v_{\theta}(z_t, t) = (\text{net}_{\theta}(z_t, t) - z_t) / (1 - t)$.

Alg. 1 shows the pseudo-code of a training step, and Alg. 2 is that of a sampling step (Euler solver; can be extended to Heun or other solvers). For brevity, class conditioning and CFG [22] are omitted, but both follow standard practice. To prevent zero division in $1/(1-t)$, we clip its denominator (by default, 0.05) whenever computing this division.

4.4. “Just Advanced” Transformers

The strength of a general-purpose Transformer [66] is partly in that, when its design is decoupled from the specific task, it can benefit from architectural advances developed in other applications. This property underpins the advantage of formulating diffusion with a task-agnostic Transformer.

Following [73], we incorporate popular general-purpose improvements⁵: SwiGLU [54], RMSNorm [75], RoPE [62], qk-norm [19], all of which were originally developed for language models. We also explore in-context class conditioning: but unlike original ViT [13] that appends one class token to the sequence, we appends multiple such tokens (by default, 32; see appendix), following [35]. Tab. 4 reports the effects of these components.

⁵Our baselines in previous sections all use SwiGLU+RMSNorm. Removing them has a slight degradation: FID goes from 7.48 to 7.89.

	JiT-B/16	JiT-L/16
Baseline (SwiGLU, RMSNorm)	7.48 (6.32)	-
+ RoPE, qk-norm	6.69 (5.44)	-
+ in-context class tokens	5.49 (4.37)	3.39 (2.79)

Table 4. “Just Advanced” Transformers with *general-purpose* designs. All are **JiT/16** for ImageNet 256×256, with bottleneck patch embedding (128-d, Fig. 4), evaluated by FID-50K. Settings: 200 epochs, with CFG (and with CFG interval [33] in brackets).

resolution	model	len	patch dim	hiddens	params	Gflops	FID
256×256	JiT-B/16	256	768	768	131	25	4.37
512×512	JiT-B/32	256	3072	768	133	26	4.64
1024×1024	JiT-B/64	256	12288	768	141	30	4.82

Table 5. **ImageNet 1024×1024 with JiT-B/64**. All entries have roughly the same number of parameters and compute. Settings: if not specified here, the same as Tab. 4 (all are with CFG interval).

256×256	200-ep	600-ep	512×512	200-ep	600-ep
JiT-B/16	4.37	3.66	JiT-B/32	4.64	4.02
JiT-L/16	2.79	2.36	JiT-L/32	3.06	2.53
JiT-H/16	2.29	1.86	JiT-H/32	2.51	1.94
JiT-G/16	2.15	1.82	JiT-G/32	2.11	1.78

Table 6. **Scalability on ImageNet 256×256 and 512×512**, evaluated by FID-50K. All models have the *same* sequence length of 16×16, and thus the models at 512 resolution have nearly the same compute as their 256 counterparts. Settings: the same as Tab. 5.

5. Comparisons

High-resolution generation on pixels. In Tab. 5, we further report our *base*-size model (JiT-B) on ImageNet at resolutions 512 and even 1024. We use patch sizes *proportional* to image sizes, and therefore the sequence length at different resolutions remains the same. The per-patch dimension can be as high as 3072 or 12288, and *none* of the common models would have sufficient hidden units.

Tab. 5 shows that our models perform decently across resolutions. All models have similar numbers of parameters and computational cost, which only differ in the input/output patch embeddings. Our method does not suffer from the curse of observed dimensionalities.

Scalability. A core goal of *decoupling* the Transformer design with the task is to leverage the potential for scalability. Tab. 6 provides results on ImageNet 256 and 512 with four model sizes (note that at resolution 512, none of these models have more hidden units than the patch dimension). The model sizes and flops are shown in Tab. 7 and 8: our model at resolution 256 has similar cost as its counterpart at 512. Our approach benefits from scaling.

Interestingly, the FID difference between resolution 256 and 512 becomes smaller with larger models. For JiT-G, the FID at 512 resolution is even lower. For very large models on ImageNet, FID performance largely depends on overfitting, and denoising at 512 resolution poses a more challenging task, making it less susceptible to overfitting.



Figure 5. **Qualitative Results.** Selected examples on ImageNet 512×512 using JiT-H/32. More uncured results are in appendix.

Reference results from previous works. As a reference, we compare with previous results in Tab. 7 and 8. We mark the *pre-training* components involved for each method. Compared with other pixel-based methods, ours is purely driven by *plain*, general-purpose Transformers. Our models are *compute-friendly* and avoid the *quadratic* scaling of expense with doubled resolution (see flops in Tab. 8).

Our approach does not use extra losses or pre-training, which may lead to further gains (an example is in appendix). These directions are left for future work.

6. Discussion and Conclusion

Noise is inherently different from natural data. Over the years, the development of diffusion models has focused primarily on probabilistic formulations, while paying less attention to the capabilities (and limitations) of the neural networks used. However, neural networks are not infinitely capable, and they can better use their capacity to model data rather than noise. Under these perspectives, our findings on x -prediction are, in hindsight, a natural outcome.

Our work adopts a minimalist and self-contained design. By reducing domain-specific inductive biases, we hope our approach can generalize to other domains where tokenizers are difficult to obtain. This property is particularly desirable for scientific applications that involve raw, high-

ImgNet 256×256	pre-training			params	Gflops	FID↓	IS↑
	token	perc.	self-sup.				
Latent-space Diffusion							
DiT-XL/2 [46]	SD-VAE	VGG	-	675+49M	119	2.27	278.2
SiT-XL/2 [40]	SD-VAE	VGG	-	675+49M	119	2.06	277.5
REPA [74], SiT-XL/2	SD-VAE	VGG	DINOv2	675+49M	119	1.42	305.7
LightningDiT-XL/2 [73]	VA-VAE	VGG	DINOv2	675+49M	119	1.35	295.3
DDT-XL/2 [71]	SD-VAE	VGG	DINOv2	675+49M	119	1.26	310.6
RAE [78], DiT ^{DH} -XL/2	RAE	VGG	DINOv2	839+415M	146	1.13	262.6
Pixel-space (non-diffusion)							
JetFormer [65]	-	-	-	2.8B	-	6.64	-
FractalMAR-H [36]	-	-	-	848M	-	6.15	348.9
Pixel-space Diffusion							
ADM-G [12]	-	-	-	554M	1120	4.59	186.7
RIN [28]	-	-	-	410M	334	3.42	182.0
SiD [25], UViT/2	-	-	-	2B	555	2.44	256.3
VDM++, UViT/2	-	-	-	2B	555	2.12	267.7
SiD2 [26], UViT/2	-	-	-	N/A	137	1.73	-
SiD2 [26], UViT/1	-	-	-	N/A	653	1.38	-
PixelFlow [6], XL/4	-	-	-	677M	2909	1.98	282.1
PixNerd [70], XL/16	-	-	DINOv2	700M	134	2.15	297
JiT-B/16	-	-	-	131M	25	3.66	275.1
JiT-L/16	-	-	-	459M	88	2.36	298.5
JiT-H/16	-	-	-	953M	182	1.86	303.4
JiT-G/16	-	-	-	2B	383	1.82	292.6

Table 7. **Reference results on ImageNet 256×256.** FID [21] and IS [53] of 50K samples are evaluated. The “pre-training” columns list the external models required to obtain the results (note that the perceptual loss [77] uses a pre-trained VGG classifier [56]). The parameters include the generator and tokenizer decoder (used at inference-time), but exclude other pre-trained components. The Giga-flops are measured for a single forward pass (not counting the tokenizer) and are roughly proportional to the computational cost of an iteration during both training and inference (for the multi-scale method [6], we measure the finest level).

ImgNet 512×512	pre-training			params	Gflops	FID↓	IS↑
	token	perc.	self-sup.				
<i>Latent-space Diffusion</i>							
DiT-XL/2 [46]	SD-VAE	VGG	-	675+49M	525	3.04	240.8
SiT-XL/2 [40]	SD-VAE	VGG	-	675+49M	525	2.62	252.2
REPA [74], SiT-XL/2	SD-VAE	VGG	DINOv2	675+49M	525	2.08	274.6
DDT-XL/2 [71]	SD-VAE	VGG	DINOv2	675+49M	525	1.28	305.1
RAE [78], DiT ^{DH} -XL/2	RAE	VGG	DINOv2	839+415M	642	1.13	259.6
<i>Pixel-space Diffusion</i>							
ADM-G [12]	-	-	-	559M	1983	7.72	172.7
RIN [28]	-	-	-	320M	415	3.95	216.0
SiD [25] , UViT/4	-	-	-	2B	555	3.02	248.7
VDM++, UViT/4	-	-	-	2B	555	2.65	278.1
SiD2 [26], UViT/4	-	-	-	N/A	137	2.19	-
SiD2 [26], UViT/2	-	-	-	N/A	653	1.48	-
PixNerd [70], XL/16	-	-	DINOv2	700 M	583	2.84	245.6
JiT-B/32	-	-	-	133M	26	4.02	271.0
JiT-L/32	-	-	-	462M	89	2.53	299.9
JiT-H/32	-	-	-	956M	183	1.94	309.1
JiT-G/32	-	-	-	2B	384	1.78	306.8

Table 8. **Reference results on ImageNet 512×512.** JiT has an *aggressive* patch size and can use *small* compute to achieve strong results. Notations are similar to Tab. 7.

dimensional natural data. We envision that the general-purpose “Diffusion + Transformer” paradigm will be a potential foundation in other areas.

A. Implementation Details

Our implementation closely follows the public codebases of DiT [46] and SiT [40]. Our configurations are summarized in Tab. 9. We describe the details as follows.

Time distribution. Following [15], during training, we adopt the logit-normal distribution over t [15]: $\text{logit}(t) \sim \mathcal{N}(\mu, \sigma^2)$. Specifically, we sample $s \sim \mathcal{N}(\mu, \sigma^2)$ and let $t = \text{sigmoid}(s)$. The hyper-parameter μ determines the noise level (see Tab. 3), and by default we set $\mu = -0.8$ on ImageNet at resolution 256 (or 512, 1024), and fix σ as 0.8.

ImageNet 512×512 and 1024×1024. We adopt JiT/32 (*i.e.*, a patch size of 32) on ImageNet 512×512. The model leads to a sequence of $256 = 16 \times 16$ patches, the same as JiT/16 on ImageNet 256×256. As such, JiT/32 only differs from JiT/16 in the input/output patch dimension, increasing from 768-d to 3072-d per patch; all other computations and costs are exactly the same.

To reuse the exact same recipe from ImageNet 256×256, for 512×512 images we rescale the magnitude of the noise ϵ by $2\times$: that is, $\epsilon \sim \mathcal{N}(0, 2^2 \mathbf{I})$. This simple modification approximately maintains the signal-to-noise ratio (SNR) between the 256×256 and 512×512 resolutions [25, 7, 26]. No other changes to the ImageNet 256×256 configuration are required or applied.

For ImageNet 1024×1024, we use the model JiT/64 and scale the noise ϵ by $4\times$. No other change is needed.

In-context class conditioning. Standard DiT [46] performs class conditioning through adaLN-Zero. In Tab. 4, we further explore in-context class-conditioning.

Specifically, following ViT [13], one can prepend a class token to the sequence of patches. This is referred to as “in-context conditioning” in DiT [46]. Note that we use in-context conditioning jointly with the default adaLN-Zero conditioning, unlike DiT. In addition, following MAR [35], we further prepend multiple such tokens to the sequence. These tokens are repeated instances of the same class token, with different positional embeddings added. We prepend 32 such tokens. Moreover, rather than prepending these tokens to the Transformer’s input, we find that prepending them at later blocks can be beneficial (see “in-context start block” in Tab. 9). Tab. 4 shows that our implementation of in-context conditioning improves FID by ~ 1.2 .

Dropout and early stop. We apply dropout [61] in JiT-H and G to mitigate the risk of overfitting. Specifically, we apply dropout to the middle half of the Transformer blocks. For Transformer blocks with dropout, we apply it to both the attention block and the MLP block.

As the G-size models still tend to overfit under our current dropout setting, we apply early stopping when the monitored FID begins to degrade. This occurs at around 320 epochs for both JiT-G/16 and JiT-G/32.

	JiT-B	JiT-L	JiT-H	JiT-G
architecture				
depth	12	24	32	40
hidden dim	768	1024	1280	1664
heads	12	16	16	16
image size	256 (other settings: 512, or 1024)			
patch size	image_size / 16			
bottleneck	128 (B/L), 256 (H/G)			
dropout	0 (B/L), 0.2 (H/G)			
in-context class tokens	32 (if used)			
in-context start block	4	8	10	10
training				
epochs	200 (ablation), 600			
warmup epochs [17]	5			
optimizer	Adam [31], $\beta_1, \beta_2 = 0.9, 0.95$			
batch size	1024			
learning rate	2e-4			
learning rate schedule	constant			
weight decay	0			
ema decay	{0.9996, 0.9998, 0.9999}			
time sampler	$\text{logit}(t) \sim \mathcal{N}(\mu, \sigma^2)$, $\mu = -0.8$, $\sigma = 0.8$			
noise scale	$1.0 \times \text{image_size} / 256$			
clip of $(1 - t)$ in division	0.05			
class token drop (for CFG)	0.1			
sampling				
ODE solver	Heun [20]			
ODE steps	50			
time steps	linear in [0.0, 1.0]			
CFG scale sweep range [22]	[1.0, 4.0]			
CFG interval [33]	[0.1, 1] (if used)			

Table 9. Configurations of experiments.

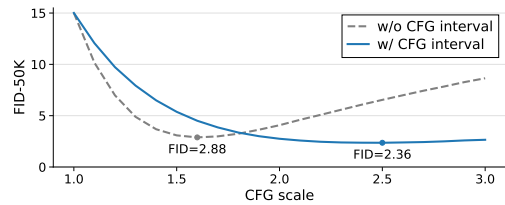


Figure 6. The influence of CFG, without and with CFG interval (for JiT-L/16 on ImageNet 256×256, 600 epochs).

EMA and CFG. Our study covers a wide range of configurations, including variations in loss and prediction spaces, model sizes, and architectural components. The optimal values of the CFG scale [22] and EMA (exponential moving average) decay vary from case to case, and fixing them may lead to incomplete or misleading observations. Since maintaining these hyperparameter configurations is relatively expensive, we strive to adopt the optimal values.

Specifically, for the CFG scale ω [22], we determine the optimal value by searching over a range of candidate scales at inference time, as is common practice in existing work. For EMA decays, we maintain multiple copies of the moving-averaged parameters during training, which introduces a negligible computational overhead. To avoid memory overhead, different EMA copies can be stored on separate devices (*e.g.*, GPUs). As such, both the CFG scale

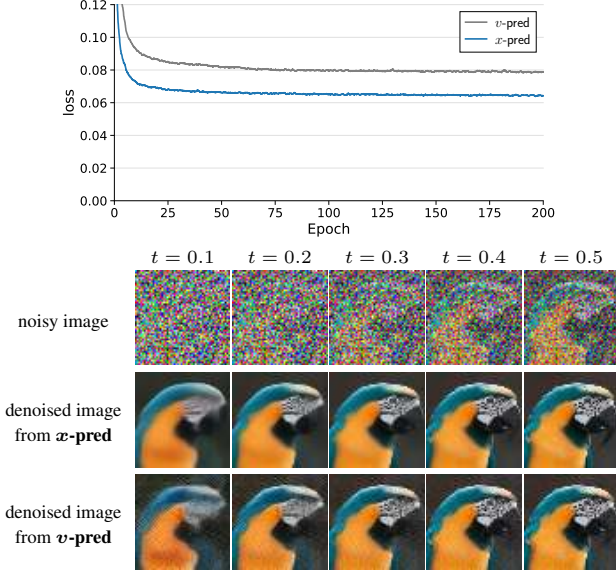


Figure 7. **(Top): Training loss** of x - and v -prediction, using the same loss space of v -loss (Tab. 2(a), third row). We plot the loss averaged per pixel per channel. **(Bottom): Denoised images** from x - and v -prediction, where v -prediction’s denoised output is visualized according to Tab. 1(c)(1). The denoised image from v -prediction has noticeable artifacts, as reflected by the higher loss.

and EMA decay are essentially inference-time decisions.

Our CFG scale candidates range from 1.0 to 4.0 with a step size of 0.1. The influence of CFG is shown in Fig. 6 for a JiT-L/16 model. Our EMA decay candidates are 0.9996, 0.9998, and 0.9999, evaluated with a batch size of 1024. For each model (including any one in the ablation), we search for the optimal setting using 8K samples and then apply it to evaluate 50K samples.

Evaluation. Following common practice, we evaluate the FID [21] against the ImageNet training set. We evaluate FID on 50K generated images, with 50 samples for each of the 1000 ImageNet classes. We evaluate the Inception Score (IS) [53] on the same 50K images.

B. Additional Experiments

B.1. Training Loss and Denoised Images

In Tab. 2(a), the failure of ϵ -/ v -prediction is caused by the inherent incapability of predicting a high-dimensional output from a limited-capacity network. This failure can be seen from the training loss curves.

In Fig. 7 (top), we compare the training loss curves under the same v -loss, defined as $\mathcal{L} = \mathbb{E} \|v_\theta(z_t, t) - v\|^2$, using v -prediction (*i.e.*, $v_\theta = \text{net}_\theta$) versus x -prediction (*i.e.*, $v_\theta = (\text{net}_\theta - z_t)/(1-t)$). Since the loss is computed in the same space and only the parameterization differs, comparing the loss values is legitimate.

Fig. 7 (top) shows that v -prediction yields substantially higher loss values (about 25%) than x -prediction, even though v -prediction appears to be the “native” parameterization for the v -loss. This comparison indicates that the task of x -prediction is inherently *easier*, as the data lie on a low-dimensional manifold. We also observe that ϵ -prediction (not shown here) has about $3\times$ higher loss and is unstable, which may explain its failure in Tab. 2(a).

Fig. 7 (bottom) compares the *denoised* images corresponding to the two training curves. For x -prediction, the denoised image is simply $x_\theta = \text{net}_\theta$; for v -prediction, the denoised image is $x_\theta = (1-t)\text{net}_\theta + z_t$ with $v_\theta = \text{net}_\theta$ (see Tab. 1(c)(1)). The *higher* loss of v -prediction in Fig. 7 (top) can be reflected by its noticeable *artifacts* in Fig. 7 (bottom).

Note that the artifact in Fig. 7 (bottom) is that of a *single* denoising step. In the generation process, this error can accumulate in the multi-step ODE solver, which leads to the catastrophic failure in Tab. 2(a).

B.2. Pre-conditioner

In EDM [29], an extra “pre-conditioner” is applied to wrap the direct output of the network. Using the notation of our paper, the pre-conditioner formulation can be written as: $x_\theta(z_t, t) = c_{\text{skip}} \cdot z_t + c_{\text{out}} \cdot \text{net}_\theta(z_t, t)$, where c_{skip} and c_{out} are pre-defined coefficients. This equation suggests that *unless* $c_{\text{skip}} \equiv 0$, the network output in a pre-conditioner must *not* perform x -prediction. And according to the manifold assumption, this formulation should not remedy the issue we consider, as we examine next.

Formulation of pre-conditioners. Given the definition of pre-conditioner, we can rewrite the “pre-conditioned x -prediction” as:

$$\begin{cases} x_\theta = c_{\text{skip}} \cdot z_t + c_{\text{out}} \cdot \text{net}_\theta \\ \epsilon_\theta = (z_t - tx_\theta)/(1-t) \\ v_\theta = (x_\theta - z_t)/(1-t) \end{cases} \quad (7)$$

Accordingly, the objective in Eq. (6) (v -loss) is written as:

$$\mathcal{L} = \mathbb{E}_{t, x, \epsilon} \|v_\theta(z_t, t) - v\|^2, \quad (8)$$

$$\begin{aligned} \text{where: } v_\theta(z_t, t) &= (x_\theta(z_t, t) - z_t)/(1-t) \\ \text{and } x_\theta(z_t, t) &= c_{\text{skip}} \cdot z_t + c_{\text{out}} \cdot \text{net}_\theta(z_t, t). \end{aligned}$$

Comparing with Eq. (6), the only difference is in how we compute x_θ from the network.

As EDM [29] uses a variance-exploding schedule (that is, $z_t = x + \sigma_t \epsilon$) and we use a (roughly) variance-preserving version (that is, $z_t = tx + (1-t)\epsilon$), a fully equivalent conversion is impossible. To have a pre-conditioner in our case, we rewrite our version as $\frac{1}{t}z_t = x + \frac{1-t}{t}\epsilon$. As such, we set $\sigma_t = \frac{1-t}{t}$, which is the noise added to unscaled images, similar to EDM’s. With this, we can rewrite the coefficients defined by EDM [29] as: $c_{\text{skip}} = \frac{1}{t} \frac{\sigma_{\text{data}}^2}{\sigma_{\text{data}}^2 + \sigma_t^2}$ and

	$\mathbf{x}$ -pred	pre-conditioned predictions	
		EDM-style	linear
	$c_{\text{skip}} = 0$	$c_{\text{skip}} = \frac{1}{t} \frac{\sigma_{\text{data}}^2}{\sigma_{\text{data}}^2 + \sigma_t^2}$	$c_{\text{skip}} = t$
	$c_{\text{out}} = 1$	$c_{\text{out}} = \frac{\sigma_{\text{data}} \sigma_t}{\sqrt{\sigma_{\text{data}}^2 + \sigma_t^2}}$	$c_{\text{out}} = 1 - t$
$\mathbf{x}$ -loss	10.14	28.94	39.50
ϵ -loss	10.45	72.05	67.56
v -loss	8.62	35.49	46.25

Table 10. **Comparisons with pre-conditioners** (FID-50K, ImageNet 256, JiT-B/16). The settings are the same as Tab. 2 (a).

FID-50K	JiT-B/16	JiT-L/16
v -loss only	4.37	2.79
w/ cls loss	4.14	2.50

Table 11. **Exploration: additional classification loss.** We do *not* use this loss in any other experiments. Settings: ImageNet 256×256, 200-ep, with CFG interval.

$c_{\text{out}} = \frac{\sigma_{\text{data}} \sigma_t}{\sqrt{\sigma_{\text{data}}^2 + \sigma_t^2}}$ where σ_{data} is the data standard deviation set as 0.5 [29]. We choose $c_{\text{in}} = t$ ($= \frac{1}{\sigma_t + 1}$), so that the direct input to the network (*i.e.*, $c_{\text{in}} \cdot \frac{1}{t} z_t$) is still z_t . It can be shown that *only* when $t \rightarrow 0$, we have: $\sigma_t \rightarrow +\infty$, $c_{\text{skip}} \rightarrow 0$, $c_{\text{out}} \rightarrow 1$, which approaches $\mathbf{x}$ -prediction. We also consider a simpler *linear* pre-conditioner: $c_{\text{skip}} = t$ and $c_{\text{out}} = 1 - t$, which also performs $\mathbf{x}$ -prediction *only* when $t = 0$.

Results of pre-conditioners. Tab. 10 shows that the pre-conditioned versions all fail catastrophically, suggesting that deviating from $\mathbf{x}$ -prediction is not desired in high-dimensional spaces. Interestingly, the pre-conditioned versions are much better than ϵ - v -prediction in Tab. 2(a). We hypothesize that this is because they are more similar to $\mathbf{x}$ -prediction when $t \rightarrow 0$, which alleviates this issue.

B.3. Exploration: Classification Loss

Our paper focuses on a minimalist design, and we intentionally do *not* use any extra loss. However, we note that latent-based methods [49] typically rely on tokenizers trained with *adversarial* and *perceptual* losses, and thus their generation process is not purely driven by diffusion. Next, we discuss a simple extension on our pixel-based models with an additional classification loss.

Formally, we attach a classifier head after a specific Transformer block (the 4th in JiT-B and the 8th in JiT-L). The classifier consists of global average pooling followed by a linear layer, and a cross-entropy loss is applied for the 1000-class ImageNet classification task. This classification loss $\mathcal{L}_{\text{cls}}$ is scaled by a weight λ (*e.g.*, 100) and added to the ℓ_2 -based (*i.e.*, element-wise sum of squared errors) regression loss. To prevent label leakage, we disable class conditioning for all layers before the classifier head. We note that this modification remains minimal and does not rely on any pre-trained classifier, unlike the perceptual loss [76].

This minor modification leads to a decent improvement, as shown in Tab. 11. This exploration suggests further potential for combining our simple method with additional loss terms, which we leave for future work.

Despite the improvement, we do *not* use this or any additional loss in the other experiments presented in this paper.

B.4. Cross-resolution Generation

A model trained at one resolution can be applied to another by simply downsampling or upsampling the generated images. We refer to this as *cross-resolution* generation. In our setup, JiT/16 at 256 and JiT/32 at 512 have comparable parameters and compute, making their cross-resolution comparison meaningful. The results are in Tab. 12

	FID@256		FID@512
JiT-G/16@256	1.82	JiT-G/16@256, $\uparrow$ 512	2.45
JiT-G/32@512, $\downarrow$ 256	1.84	JiT-G/32@512	1.78

Table 12. **Cross-resolution Generation** (noted in gray), using JiT-G/16 trained at resolution 256 and JiT-G/32 trained at 512, followed by upsampling ($\uparrow$) or downsampling ($\downarrow$). All entries have similar parameters and flops (see Tab. 7 and 8).

Downsampling the images generated by the 512 model to 256 resolution yields a decent FID@256 of 1.84. This result remains competitive when compared with the 256-resolution expert (FID@256 of 1.82), while maintaining a similar computational cost and gaining the additional ability to generate at 512 resolution.

On the other hand, *upsampling* the images generated by the 256 model to 512 resolution results in a noticeably worse FID@512 of 2.45, compared with the 512-resolution expert’s FID@512 of 1.78. This degradation is caused by the loss of higher-frequency details due to upsampling.

B.5. Additional Metrics

For completeness, we report precision and recall [32] on ImageNet 256×256 in Tab. 13, compared with the commonly used baselines of DiT and SiT, and the latest RAE:

	FID↓	IS↑	Prec↑	Rec↑
DiT-XL/2 [46]	2.27	278.2	0.83	0.57
SiT-XL/2 [40]	2.06	277.5	0.82	0.59
RAE [78], DiT ^{DH} -XL/2	1.13	262.6	0.78	0.67
JiT-B/16	3.66	275.1	0.82	0.50
JiT-L/16	2.36	298.5	0.80	0.59
JiT-H/16	1.86	303.4	0.78	0.62
JiT-G/16	1.82	292.6	0.79	0.62

Table 13. Precision and recall on ImageNet 256×256.

C. Qualitative Results

In Fig. 8 to 11, we provide additional *uncurated* examples on ImageNet 256×256.

Acknowledgements. We thank Google TPU Research Cloud (TRC) for granting us access to TPUs, and the MIT ORCD Seed Fund Grants for supporting GPU resources.

References

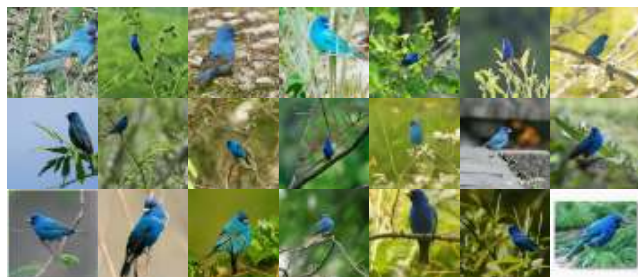
- [1] Michael Samuel Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *ICLR*, 2023.
- [2] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. Deep variational information bottleneck. In *ICLR*, 2017.
- [3] Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46(2):255–308, 2009.
- [4] Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien, editors. *Semi-Supervised Learning*. MIT Press, Cambridge, MA, USA, 2006.
- [5] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.
- [6] Shoufa Chen, Chongjian Ge, Shilong Zhang, Peize Sun, and Ping Luo. PixelFlow: Pixel-space generative models with flow. *arXiv:2504.07963*, 2025.
- [7] Ting Chen. On the importance of noise scheduling for diffusion models. *arXiv:2301.10972*, 2023.
- [8] Xinlei Chen, Zhuang Liu, Saining Xie, and Kaiming He. Deconstructing denoising diffusion models for self-supervised learning. In *ICLR*, 2025.
- [9] Kostadin Dabov, Alessandro Foi, Vladimir Katkovnik, and Karen Egiazarian. Image denoising by sparse 3-D transform-domain collaborative filtering. *IEEE Transactions on image processing*, 16(8):2080–2095, 2007.
- [10] Mauricio Delbracio and Peyman Milanfar. Inversion by direct iteration: An alternative to denoising diffusion for image restoration. *Transactions on Machine Learning Research*, 2023.
- [11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [12] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.
- [13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [14] Michael Elad and Michal Aharon. Image denoising via sparse and redundant representations over learned dictionaries. *IEEE Transactions on Image processing*, 15(12):3736–3745, 2006.
- [15] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, and Robin Rombach. Scaling rectified flow Transformers for high-resolution image synthesis. In *ICML*, 2024.
- [16] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *NeurIPS*, 2014.
- [17] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*, 2017.
- [18] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv:2509.24527*, 2025.
- [19] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for Transformers. In *Findings of EMNLP*, 2020.
- [20] Karl Heun. Neue methoden zur approximativen integration der differentialgleichungen einer unabhängigen veränderlichen. *Z. Math. Phys*, 45:23–38, 1900.
- [21] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *NeurIPS*, 2017.
- [22] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshops*, 2021.
- [23] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [24] Jonathan Ho, Ajay Jain, and Pieter Abbeel. DDPM github repo. `L155, diffusion.utils.2.py`, 2020.
- [25] Emiel Hoogeboom, Jonathan Heek, and Tim Salimans. simple diffusion: End-to-end diffusion for high resolution images. *ICML*, 2023.
- [26] Emiel Hoogeboom, Thomas Mensink, Jonathan Heek, Kay Lamerigts, Ruiqi Gao, and Tim Salimans. Simpler Diffusion (SiD2): 1.5 FID on ImageNet512 with pixel-space diffusion. In *CVPR*, 2025.
- [27] Ahmed Imtiaz Humayun, Ibtihel Amara, Cristina Vasconcelos, Deepak Ramachandran, Candice Schumann, Junfeng He, Katherine Heller, Golnoosh Farnadi, Negar Rostamzadeh, and Mohammad Havaei. What secrets do your manifolds hold? understanding the local geometry of generative models. In *ICLR*, 2025.
- [28] Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative generation. In *ICML*, 2023.
- [29] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *NeurIPS*, 2022.
- [30] Diederik Kingma and Ruiqi Gao. Understanding diffusion objectives as the ELBO with simple data augmentation. In *NeurIPS*, 2023.
- [31] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [32] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. *NeurIPS*, 2019.
- [33] Tuomas Kynkäänniemi, Miika Aittala, Tero Karras, Samuli Laine, Timo Aila, and Jaakko Lehtinen. Applying guidance in a limited interval improves sample and distribution quality in diffusion models. In *NeurIPS*, 2024.

- [34] Jiachen Lei, Keli Liu, Julius Berner, Haiming Yu, Hongkai Zheng, Jiahong Wu, and Xiangxiang Chu. Advancing end-to-end pixel space generative modeling via self-supervised pre-training. *arXiv:2510.12586*, 2025.
- [35] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive image generation without vector quantization. In *NeurIPS*, 2024.
- [36] Tianhong Li, Qinyi Sun, Lijie Fan, and Kaiming He. Fractal generative models. *arXiv:2502.17437*, 2025.
- [37] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023.
- [38] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *ICLR*, 2023.
- [39] Gabriel Loaiza-Ganem, Brendan Leigh Ross, Rasa Hosseinzadeh, Anthony L Caterini, and Jesse C Cresswell. Deep generative models through the lens of the manifold hypothesis: A survey and new connections. *Transactions on Machine Learning Research*, 2024.
- [40] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. SiT: Exploring flow and diffusion-based generative models with scalable interpolant Transformers. In *ECCV*, 2024.
- [41] Alireza Makhzani and Brendan Frey. K-sparse autoencoders. *arXiv:1312.5663*, 2013.
- [42] Peyman Milanfar and Mauricio Delbracio. Denoising: a powerful building block for imaging, inverse problems and machine learning. *Philosophical Transactions A*, 383(2299): 20240326, 2025.
- [43] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.
- [44] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
- [45] Maxime Oquab et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2023.
- [46] William Peebles and Saining Xie. Scalable diffusion models with Transformers. In *ICCV*, 2023.
- [47] Javier Portilla, Vasily Strela, Martin J Wainwright, and Eero P Simoncelli. Image denoising using scale mixtures of Gaussians in the wavelet domain. *IEEE Transactions on Image processing*, 12(11):1338–1351, 2003.
- [48] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In *ICML*, 2011.
- [49] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [50] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- [51] Sam T Roweis and Lawrence K Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.
- [52] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *ICLR*, 2022.
- [53] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training GANs. *NeurIPS*, 29, 2016.
- [54] Noam Shazeer. GLU variants improve Transformer. *arXiv:2002.05202*, 2020.
- [55] Minglei Shi, Haolin Wang, Wenzhao Zheng, Ziyang Yuan, Xiaoshi Wu, Xintao Wang, Pengfei Wan, Jie Zhou, and Jiwen Lu. Latent diffusion model without variational autoencoder. *arXiv:2510.15301*, 2025.
- [56] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*, 2014.
- [57] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.
- [58] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.
- [59] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019.
- [60] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.
- [61] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [62] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced Transformer with rotary position embedding. *Neurocomputing*, 568: 127063, 2024.
- [63] Joshua B Tenenbaum, Vin de Silva, and John C Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.
- [64] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [65] Michael Tschannen, André Susano Pinto, and Alexander Kolesnikov. JetFormer: an autoregressive generative model of raw images and text. In *ICLR*, 2025.
- [66] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.
- [67] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011.
- [68] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *ICML*, 2008.
- [69] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11(12), 2010.

- [70] Shuai Wang, Ziteng Gao, Chenhui Zhu, Weilin Huang, and Limin Wang. PixNerd: Pixel neural field diffusion. *arXiv:2507.23268*, 2025.
- [71] Shuai Wang, Zhi Tian, Weilin Huang, and Limin Wang. DDT: Decoupled diffusion Transformer. *arXiv:2504.05741*, 2025.
- [72] Yutong Xie, Minne Yuan, Bin Dong, and Quanzheng Li. Diffusion model for generative image denoising. In *ICCV*, 2023.
- [73] Jingfeng Yao, Bin Yang, and Xinggang Wang. Reconstruction vs. generation: Taming optimization dilemma in latent diffusion models. In *CVPR*, 2025.
- [74] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion Transformers is easier than you think. In *ICLR*, 2025.
- [75] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.
- [76] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [77] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [78] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion Transformers with representation autoencoders. *arXiv:2510.11690*, 2025.
- [79] Daniel Zoran and Yair Weiss. From learning models of natural image patches to whole image restoration. In *ICCV*, 2011.



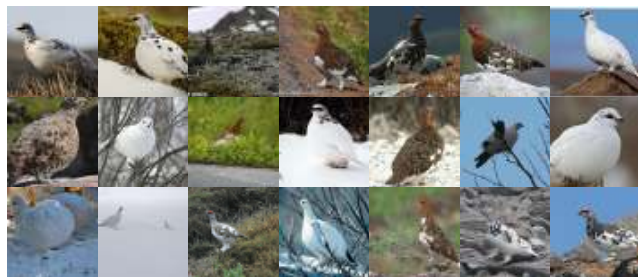
class 012: house finch, linnet, *Carpodacus mexicanus*



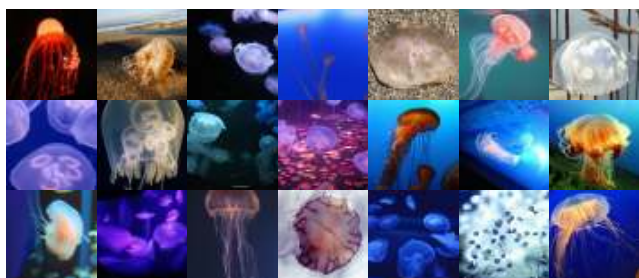
class 014: indigo bunting, indigo finch, indigo bird, *Passerina cyanea*



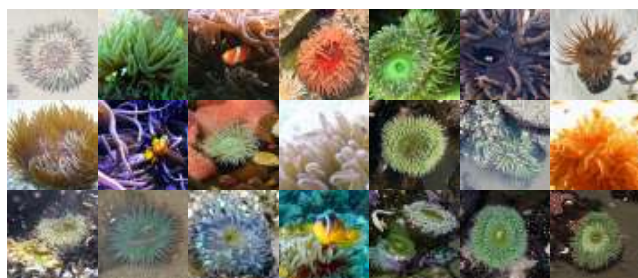
class 042: agama



class 081: ptarmigan



class 107: jellyfish



class 108: sea anemone, anemone



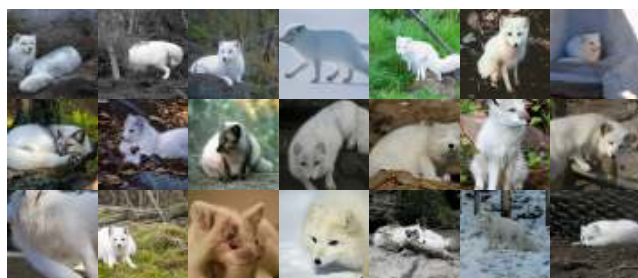
class 110: flatworm, platyhelminth



class 117: chambered nautilus, pearly nautilus, nautilus

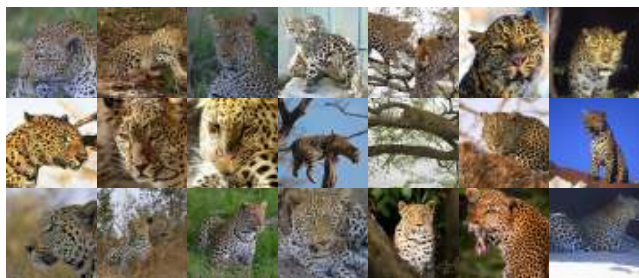


class 130: flamingo

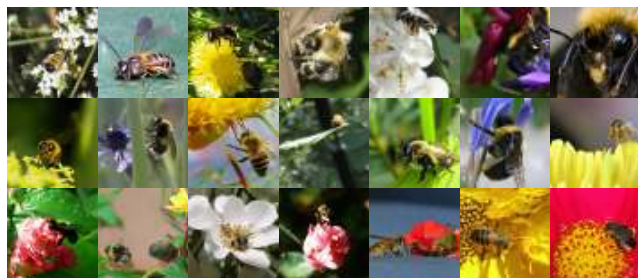


class 279: Arctic fox, white fox, *Alopex lagopus*

Figure 8. *Uncurated* samples on ImageNet 256×256 using JiT-G/16 conditioned on the specified classes. Unlike the common practice of visualizing with a higher CFG, here we show images using the CFG value (2.2) that achieves the reported FID of 1.82.



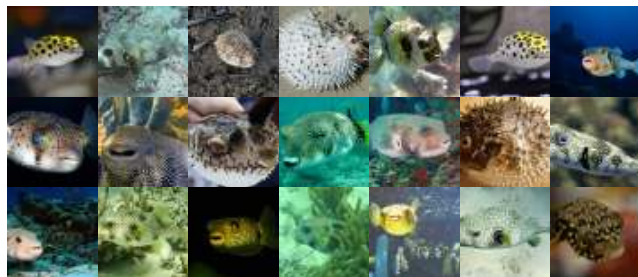
class 288: leopard, Panthera pardus



class 309: bee



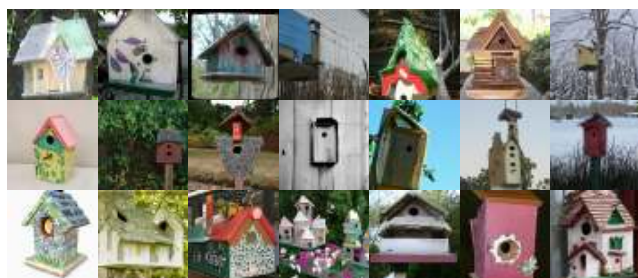
class 349: bighorn, bighorn sheep, cimarron, Rocky Mountain bighorn



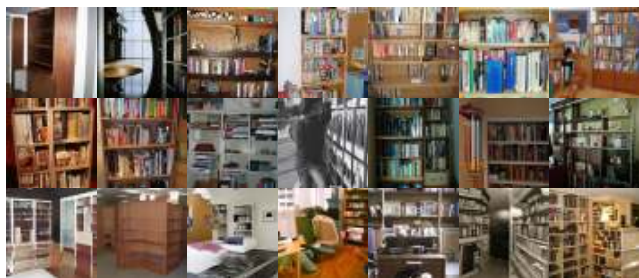
class 397: puffer, pufferfish, blowfish, globefish



class 425: barn



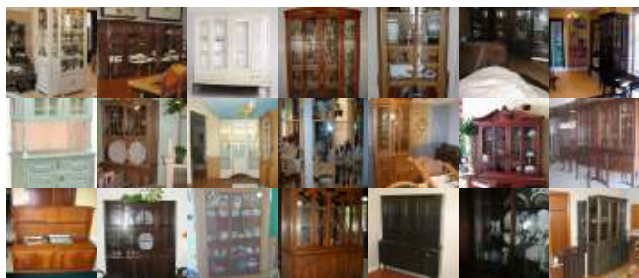
class 448: birdhouse



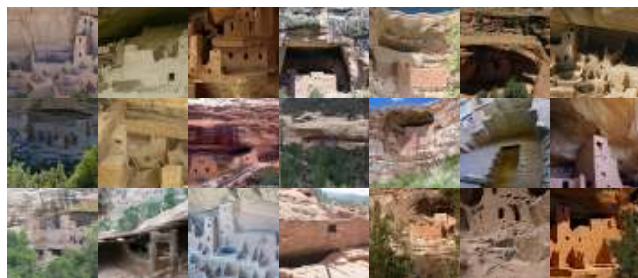
class 453: bookcase



class 458: brass, memorial tablet, plaque



class 495: china cabinet, china closet



class 500: cliff dwelling

Figure 9. *Uncurated* samples on ImageNet 256×256 using JiT-G/16 conditioned on the specified classes. Unlike the common practice of visualizing with a higher CFG, here we show images using the CFG value (2.2) that achieves the reported FID of 1.82.



class 658: mitten



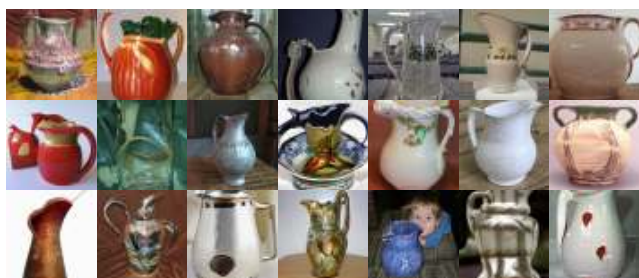
class 661: Model T



class 718: pier



class 724: pirate, pirate ship



class 725: pitcher, ewer



class 757: recreational vehicle, RV, R.V.



class 779: school bus



class 780: schooner

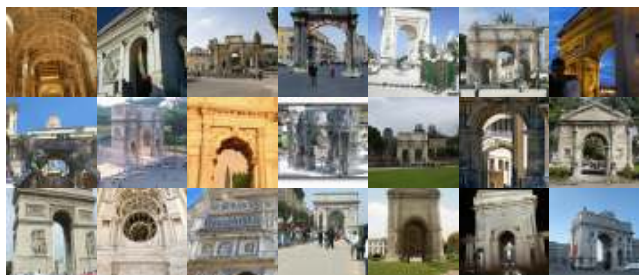


class 829: streetcar, tram, tramcar, trolley, trolley car

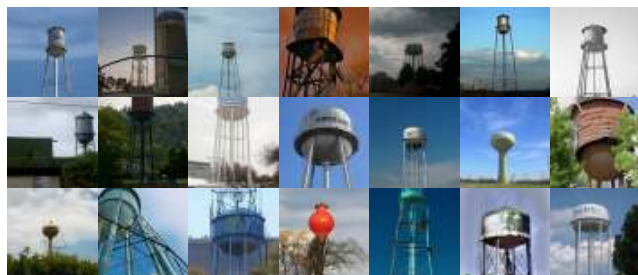


class 853: thatch, thatched roof

Figure 10. *Uncurated* samples on ImageNet 256×256 using JiT-G/16 conditioned on the specified classes. Unlike the common practice of visualizing with a higher CFG, here we show images using the CFG value (2.2) that achieves the reported FID of 1.82.



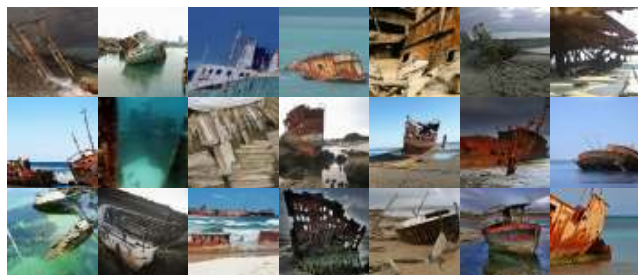
class 873: triumphal arch



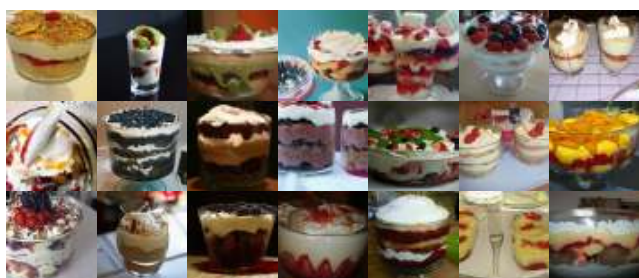
class 900: water tower



class 911: wool, woolen, woollen



class 913: wreck



class 927: trifle



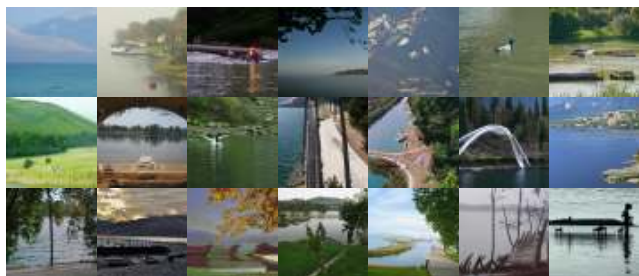
class 930: French loaf



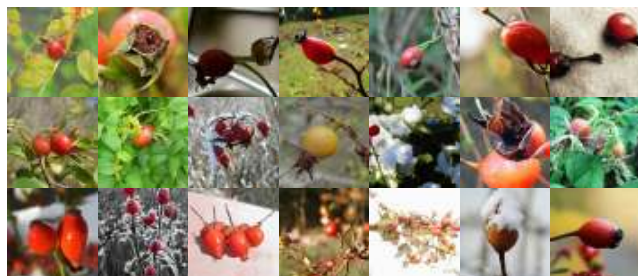
class 946: cardoon



class 947: mushroom



class 975: lakeside, lakeshore



class 989: hip, rose hip, rosehip

Figure 11. *Uncurated* samples on ImageNet 256×256 using JiT-G/16 conditioned on the specified classes. Unlike the common practice of visualizing with a higher CFG, here we show images using the CFG value (2.2) that achieves the reported FID of 1.82.